Project Plan

NileFlow: Real-Time Traffic & Transit Congestion Intelligence Platform
for Greater Cairo and Alexandria

GitHub: https://github.com/mohamed-mahmoud-de/NileFlow

1. Project Description

This project aims to design and implement an end-to-end, real-time data engineering platform that ingests live traffic conditions, public transit schedule data, and weather observations for Greater Cairo and Alexandria, processes this data using stream and batch pipelines, detects congestion and delay anomalies, and surfaces actionable insights through a live dashboard and an automated alerting channel.

The project focuses on applying advanced Data Engineering concepts such as event streaming, stream-batch hybrid architecture, real-time analytics, anomaly detection, distributed storage, workflow orchestration, containerization, and live data visualization using industry-standard tools.

2. Technologies and Tools

- Programming Language: Python
- Data Sources: Transport for Cairo (GTFS transit data), live traffic/routing API (Google Routes API or self-hosted OSRM), Open-Meteo Weather API
- Data Format: JSON, GTFS (CSV-based)
- Streaming Platform: Apache Kafka
- Stream Processing: Apache Spark Structured Streaming
- Batch Orchestration: Apache Airflow
- Relational Database: PostgreSQL (reference data: routes, stops, districts)
- Time-Series / NoSQL Database: Apache Cassandra (congestion and weather metrics)
- Search and Indexing: Elasticsearch (alert and event log search)
- Caching / Pub-Sub: Redis (real-time alert fan-out)
- Notification Channel: Discord Webhook (live congestion alerts)
- Containerization: Docker & Docker Compose
- Dashboard: Streamlit or Grafana (live map and time-series visualization)
- Version Control: Git & GitHub

3. Problem Statement

Greater Cairo's transportation network is a patchwork of formal metro and bus lines alongside informal microbus and minibus routes, with no unified real-time information layer. Commuters have no visibility into delays, and no public, data-driven view exists showing where the network is congested at any given moment. Traffic congestion in Cairo and Alexandria is consistently cited as one of the most significant daily-life challenges in Egypt, yet there is no open, real-time analytics system that combines transit, traffic, and weather data to explain how severe congestion is, where it is occurring, and why.

4. Proposed Solution

NileFlow addresses this gap by building a real-time congestion and transit-delay monitoring platform for Greater Cairo and Alexandria. The system ingests live traffic conditions, transit route and schedule data, and weather observations through Kafka, processes them using Spark Structured Streaming to compute a rolling congestion index per corridor and detect anomalies (sudden slowdowns, abnormal transit delays), and stores results across PostgreSQL, Cassandra, and Elasticsearch depending on data type. Insights are visualized on a live map-based dashboard, and severe anomalies trigger automated alerts via a Discord webhook, providing a usable tool for city planners, logistics and delivery companies, or commuters.

5. Project Objectives

- Build a real-time, event-driven data pipeline using Apache Kafka.
- Ingest live traffic, transit, and weather data from public APIs and open data sources.
- Process streaming data with Apache Spark Structured Streaming, including time-window joins.
- Compute a real-time congestion index and detect anomalies (delays, slowdowns).
- Design a multi-database storage layer (PostgreSQL, Cassandra, Elasticsearch).
- Automate batch refresh and data quality checks using Apache Airflow.
- Deliver real-time alerts through a Discord webhook integration.
- Build a live, map-based dashboard for congestion visualization.
- Produce clear documentation and a working live demo.

6. Project Milestones and Tasks

Milestone 1: Reference Data and Infrastructure Setup

Objective: Establish the static data foundation and core infrastructure.

- Acquire transit route, stop, and schedule data (GTFS from Transport for Cairo, or OpenStreetMap-derived routes as fallback).
- Design and load reference schema into PostgreSQL (routes, stops, corridors, districts).
- Set up Docker Compose skeleton: Kafka, Spark, PostgreSQL, Cassandra, Elasticsearch, Redis, Airflow.
- Verify inter-service connectivity within the container network.

Deliverables: PostgreSQL reference schema and data, working Docker Compose environment.

Milestone 2: Real-Time Data Ingestion (Kafka Producers)

Objective: Stream live traffic, weather, and transit position data into Kafka.

- Build a producer for live traffic/travel-time data per key corridor (traffic_events topic).
- Build a producer for live weather observations for Cairo and Alexandria (weather_events topic).
- Build a producer simulating real-time vehicle position pings along transit route geometries (vehicle_position_events topic).

- Validate message schemas and topic throughput.

Deliverables: Three functioning Kafka producers, sample event streams.

Milestone 3: Stream Processing and Anomaly Detection

Objective: Process streaming data to compute congestion metrics and detect anomalies.

- Implement Spark Structured Streaming jobs consuming all three Kafka topics.
- Apply time-window joins and watermarking to correlate traffic, weather, and transit data.
- Compute a rolling congestion index per corridor (current travel time vs. baseline).
- Implement anomaly detection logic for sustained high-congestion or transit-delay events.
- Write aggregated metrics to Cassandra and anomaly/alert events to Elasticsearch.

Deliverables: Working Spark Structured Streaming application, populated Cassandra and Elasticsearch indices.

Milestone 4: Orchestration, Alerting, and Dashboard

Objective: Automate batch tasks, deliver real-time alerts, and visualize results.

- Build Airflow DAGs for daily reference data refresh and rolling baseline recalculation.
- Build an Airflow DAG for data quality and pipeline freshness checks.
- Implement Redis-based pub/sub to fan out detected anomalies.
- Integrate a Discord webhook to post formatted real-time congestion alerts.
- Build a live dashboard (Streamlit or Grafana) showing a congestion heatmap, time-series trends, and an alert feed.

Deliverables: Airflow DAGs, Discord alert integration, live dashboard.

Milestone 5: Final Documentation, Demo, and Presentation

Objective: Deliver a complete, well-documented project with a working live demonstration.

- Document the end-to-end architecture, data flow, and design decisions.
- Describe database schemas across PostgreSQL, Cassandra, and Elasticsearch.
- Prepare a live demo of the full pipeline running end-to-end.
- Run analytical queries demonstrating congestion patterns and weather correlation.
- Prepare final project presentation.

Deliverables: Project documentation, GitHub repository, demo and presentation.

7. Final Project Deliverables

- End-to-end real-time traffic and transit congestion monitoring platform.
- Kafka-based streaming ingestion layer (traffic, weather, transit position data).
- Spark Structured Streaming application for congestion scoring and anomaly detection.
- Multi-database storage layer (PostgreSQL, Cassandra, Elasticsearch).
- Airflow DAGs for batch refresh and data quality monitoring.
- Discord webhook integration for real-time alerts.
- Live map-based dashboard (Streamlit or Grafana).
- Docker and Docker Compose setup for the full stack.

- Project documentation and GitHub repository.
- Final presentation.

8. Project Value

This project demonstrates advanced Data Engineering skills including real-time stream processing, event-driven architecture, multi-source data correlation, anomaly detection, polyglot persistence, workflow orchestration, and live data visualization. It addresses a concrete, widely recognized problem in Egyptian daily life and reflects real-world streaming data pipeline design used in transportation analytics, logistics, and smart-city systems, preparing the student for advanced Data Engineering and Streaming Engineering roles.